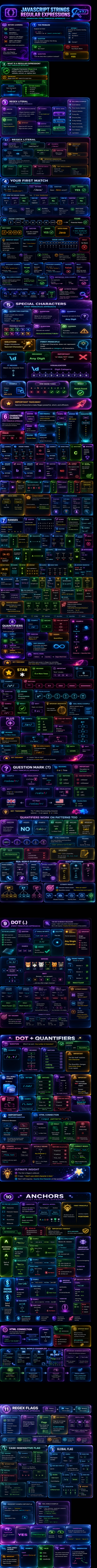
Hitesh Rathore (HR)



JAVASCRIPT STRINGS REGULAR EXPRESSIONS

FOUNDATION (FIRST PRINCIPLES APPROACH)

BEFORE LEARNING:

- match() Search for a match
- matchAll() Find all matches
- replace() with Regex Replace using a pattern
- split() with Regex Split using a pattern

we must answer one question.

QUESTION Why were Regular Expressions created?

1 THE PROBLEM

Suppose we have:

```
let text =
 "Contact me at [email protected]";
```

QUESTION Can we check whether this String contains an email?

OPTION 1 Character-by-character checking. Check:

- Letters?
- Numbers?
- @
- .
- Domain?

YES But how?

PROBLEM Too Much Work

QUESTION Can we describe a pattern instead?

ANSWER YES

2 WHAT IS A REGULAR EXPRESSION?

FIRST PRINCIPLES DEFINITION

A Regular Expression (Regex) is a pattern used to search, match, validate, extract, or replace text.

IMPORTANT Regex does NOT search for exact text.

IT SEARCHES FOR Patterns

EXAMPLE Exact Text Search ↓ "JavaScript"

PATTERN SEARCH

- Any Number
- Any Email
- Any Date
- Any Phone Number

3 REGEX LITERAL

(FIRST PRINCIPLES APPROACH)

QUESTION What is Regex?

ANSWER Regex stands for: Regular Expression

FIRST PRINCIPLES DEFINITION A Regular Expression is a pattern used to search, match, validate, extract, or replace text.

IMPORTANT Regex does NOT store text. Regex describes a pattern.

EXAMPLE Text: "I love JavaScript" Regex: Java

QUESTION What are we looking for?

ANSWER The pattern: Java

MENTAL MODEL Text → Search Rules → Match Found?

REAL WORLD EXAMPLES

- Email Validation
- Phone Number Validation
- Password Validation
- Finding URLs
- Extracting Dates
- Finding Hashtags
- Finding Usernames
- Search Functionality
- Form Validation
- Log Analysis

3.1 REGEX LITERAL

4 YOUR FIRST MATCH

5 SPECIAL CHARACTERS

6 CHARACTER CLASSES

7 RANGES

8 QUANTIFIERS

QUESTION MARK (?)

QUANTIFIERS WORK ON PATTERNS TOO

9 DOT (.)

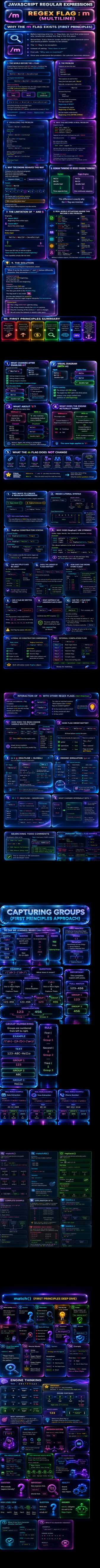
DOT + QUANTIFIERS

10 ANCHORS

11 REGEX FLAGS

CASE INSENSITIVE FLAG

GLOBAL FLAG



JAVASCRIPT REGULAR EXPRESSIONS

REGEX FLAG : m (MULTILINE)

WHY THE m FLAG EXISTS (FIRST PRINCIPLES)

- Before learning what the 'm' flag does, we must first understand why JavaScript needed this flag in the first place.
- Remember: Every feature inside JavaScript exists because some real-world problem needed to be solved.
- The 'm' flag is no exception.
- Instead of asking: "How does m work?"
- First ask: "Why was m invented?"
- Once you understand the problem, the solution becomes obvious.

1. THE WORLD BEFORE THE m FLAG

Imagine Regular Expressions did NOT have the m (multiline) flag. The regex engine sees the entire input as ONE BIG STRING.

Example:

```
Hello\nWorld\nJavaScript
```

The engine internally thinks:

```
Hello\nWorld\nJavaScript
```

Notice something. Although we see three separate lines, the regex engine initially sees only ONE continuous sequence of characters. It does NOT automatically think:

- Line 1 Hello
- Line 2 World
- Line 3 JavaScript

Instead, it thinks:

```
H e l l o \n W o r l d \n J a v a S c r i p t
```

Everything belongs to one single input. This idea is extremely important. The regex engine does NOT naturally understand the concept of "lines."

It only understands:

- Characters
- Positions
- Beginning of input
- End of input

Nothing more.

2. THE PROBLEM

Suppose we have this text:

```
Hello
World
JavaScript
```

Now suppose we want to find: Every line that starts with "World"

Naturally, a programmer might write:

```
^World
```

because they know: ^ means "Beginning"

But here comes the problem. The regex engine asks: Beginning of WHAT?

The programmer means: Beginning of THIS line.

But the engine understands: Beginning of the ENTIRE STRING.

3. VISUALIZING THE PROBLEM

4. WHY THIS BECOMES A REAL PROBLEM

Imagine a log file.

```
INFO  User Logged In
ERROR Database Failed
INFO  Request Completed
ERROR Network Timeout
```

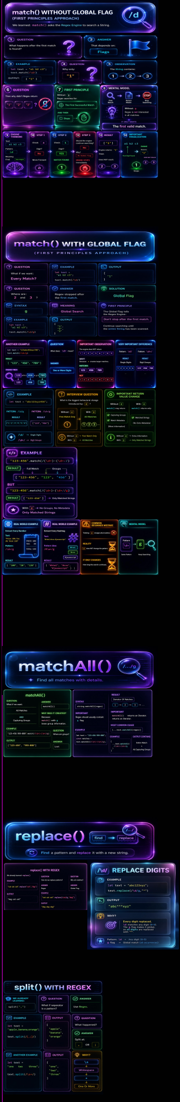
Suppose we want every line beginning with: ERROR

10. FIRST PRINCIPLES SUMMARY

CAPTURING GROUPS (FIRST PRINCIPLES APPROACH)

match() (FIRST PRINCIPLES DEEP DIVE)

ENGINE THINKING



match() WITHOUT GLOBAL FLAG

(FIRST PRINCIPLES APPROACH)

We learned: match() asks the Regex Engine to search a String.

1 QUESTION What happens after the first match is found?

2 ANSWER That depends on: Flags

3 EXAMPLE

```
let text = "a1 b2 c3";
text.match(/\d/)
```

OUTPUT: ["1"]

4 QUESTION Why only: "1"

5 OBSERVATION The String contains: 1 → 2 → 3

6 QUESTION Then why didn't Regex return: "1" "2" "3" ?

7 FIRST PRINCIPLE Without: g Regex searches for: The First Successful Match AND THEN Stops

8 MENTAL MODEL Find Match → Return Match → Stop Searching

Without: g Regex is not interested in all matches.

IT ONLY WANTS The first valid match.

9 ENGINE THINKING Text: a1 b2 c3 Pattern: \d

10 STEP 1 Check: a Digit? No Move Forward

11 STEP 2 Check: 1 Digit? YES MATCH FOUND

12 STEP 3 Should the engine continue searching? No ENGINE STOPS Immediately

13 RESULT ["1"] Engine returns: "1" Stops. Does NOT reach: 2 3

14 IMPORTANT VISUALIZATION a1 b2 c3

match() WITH GLOBAL FLAG

(FIRST PRINCIPLES APPROACH)

QUESTION What if we want: Every Match?

EXAMPLE

```
let text =
 "a1 b2 c3";
text.match(/\d/)
```

OUTPUT

```
[
  "1"
]
```

QUESTION Where are: 2 and 3 ?

ANSWER Regex stopped after the first match.

SOLUTION Global Flag

SYNTAX g

MEANING Global Search

FIRST PRINCIPLE The Global Flag tells the Regex Engine:

matchAll()

replace()

REPLACE DIGITS

split() WITH REGEX